

PHYSICAL AI FINAL PROJECT

CUPID - 사랑이 싹트는 42

터틀봇이 익명의 메시지를 직접 배달하는 오프라인 대나무숲

PROBLEM

왜 이 프로젝트인가

말 걸기 어색한 분위기 속, 터틀봇이 말을 대신 전함



“저 사람한테 말 걸고
싶었는데...”



“오늘 도움받았는데 고맙다는
말을 못 했어.”



“사실 관심있었어.”



우리는 그 말들에게 다리를 놓아준다 — 터틀봇이 직접 찾아가서

어디로 가야 하는가? 클러스터 실측 도면 기반 좌표 매핑

어떻게 전달하는가? 웹 UI → WebSocket → ROS2 → 로봇

사용 흐름

01



발신자

웹 UI에서 수신자 좌석을
고르고 익명 메시지를
작성·전송

02



터틀봇

클러스터 맵(발전 방향) 기반
자율주행으로 해당 좌석까지
직접 이동

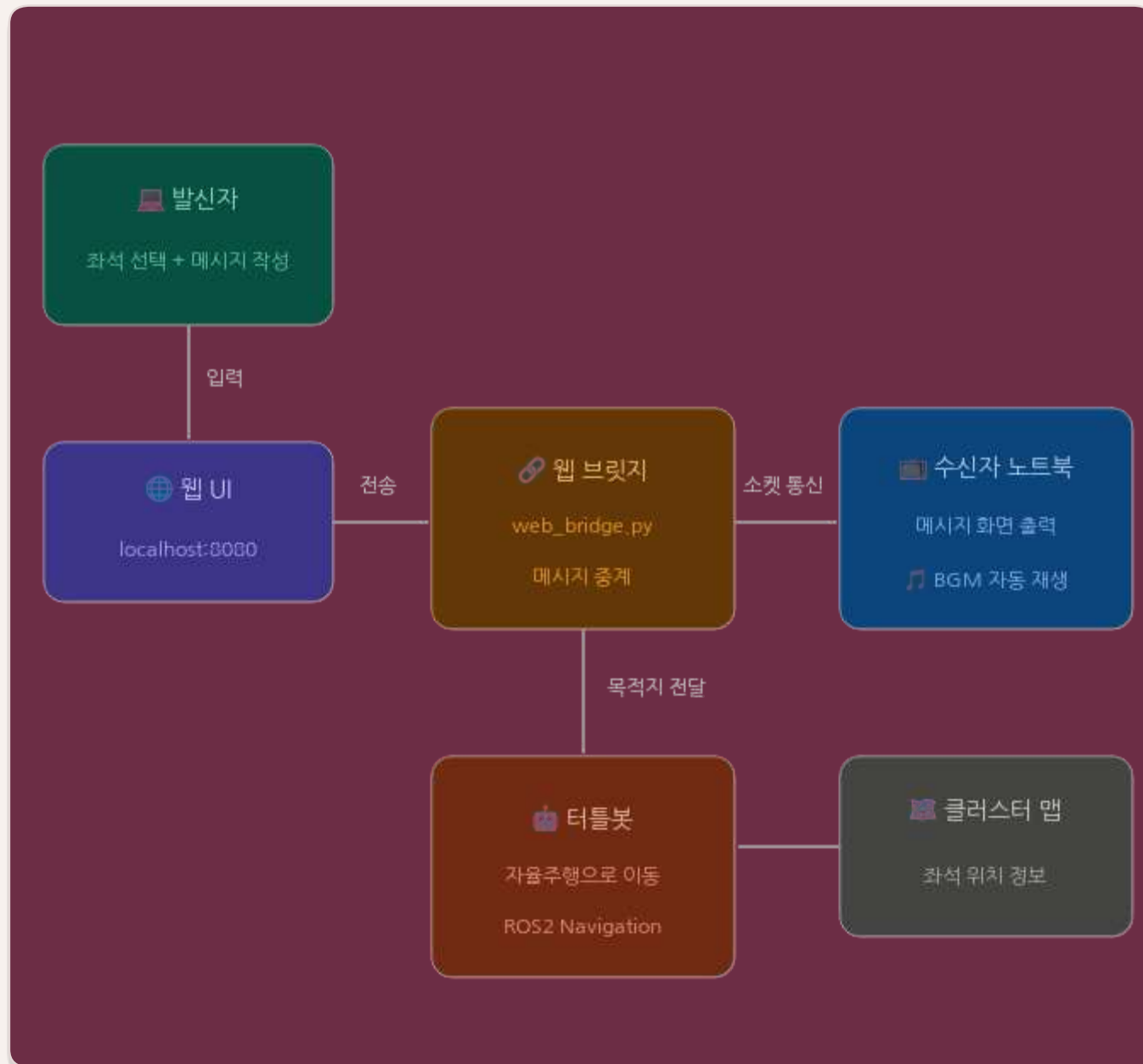
03



외부 출력

BGM 재생(외부스피커)+ 외부
디스플레이 화면(발전 방향)에
메시지 출력

시스템 아키텍처



메시지 한 통이 흐르는 길



사용자 입력

"웹소켓을 통해 c2r5s2 자리로 이동하라는 요청 수신."



브리지 → Nav2

"c2r5s2 위치로 이동하라는 명령 송신."



실시간 피드백

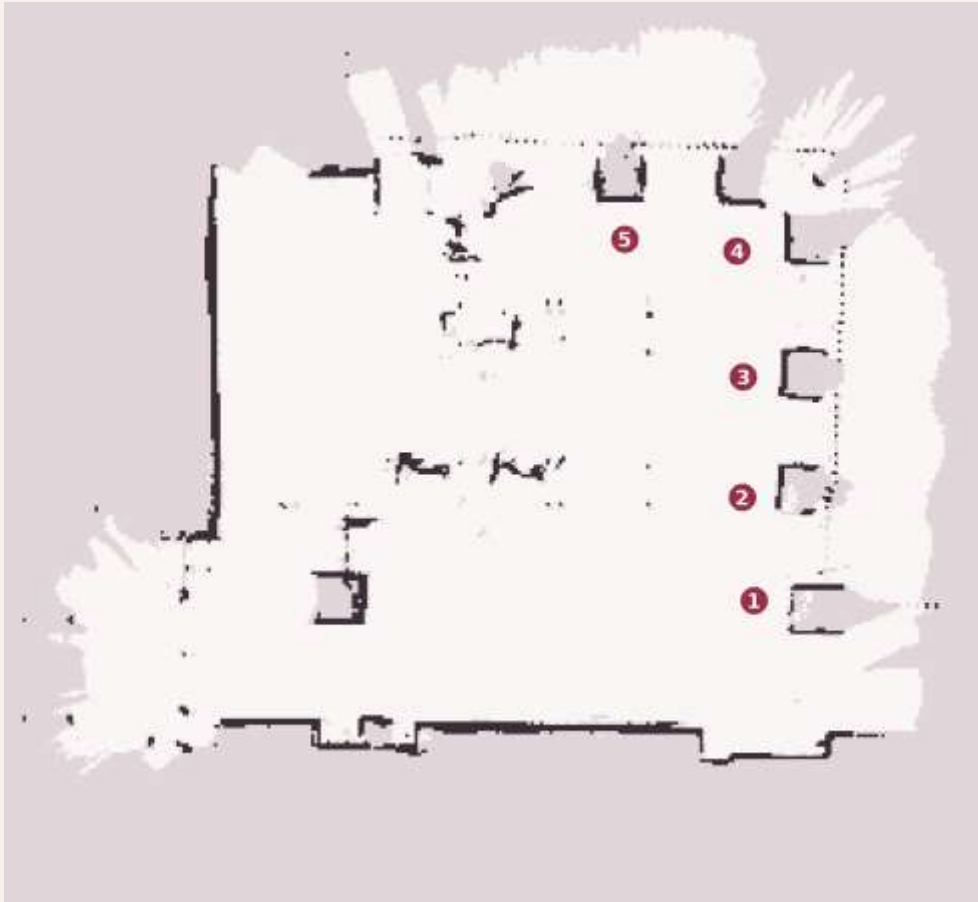
"로봇의 남은 시간과 거리를 브라우저에 보내고,
화면의 도착 예상 시간과 미니맵을 실시간 업데이트"



도착 완료

‘BGM 재생 + 도착 화면 전환’

좌석을 좌표로 — 실측 SLAM 맵



어떻게 좌표를 얻었나

- TurtleBot과 Cartographer를 이용해 클러스터 공간 지도 제작
- 터틀봇을 각 소파 위치로 직접 이동시킨 뒤, 터미널에서 현재 위치를 확인하는 명령어로 좌표를 확인
- 위치마다 좌표정보(x, y)와 방향 정보를 저장해 두었고,
- 나중에 로봇이 해당 위치로 이동할 수 있도록 사용함

```
SEATS = {  
  1: {'x': 8.46, 'y': 1.04, 'oz': -0.114, 'ow': 0.993},  
  2: {'x': 8.27, 'y': 2.68, 'oz': 0.132, 'ow': 0.991},  
  3: {'x': 8.29, 'y': 4.61, 'oz': 0.134, 'ow': 0.990},  
  4: {'x': 8.19, 'y': 6.63, 'oz': 0.758, 'ow': 0.651},  
  5: {'x': 6.40, 'y': 6.82, 'oz': 0.790, 'ow': 0.612},  
}
```

CORE IMPLEMENTATION

ROS2 ↔ 브라우저 브리지

두 세계를 잇는다: 동기적인 ROS2 spin과 비동기 WebSocket 서버



ROS2 spin

별도 thread에서 rclpy.spin() 실행 — 액션·토픽 콜백 담당



WebSocket 서버

asyncio 이벤트 루프에서 :8765 서빙 — 브라우저 양방향 연결



두 세계 연결

콜백에서 run_coroutine_threadsafe() 로 asyncio 루프에 안전하게 broadcast

\$ ros2 node list

```
/amcl  
/bt_navigator  
/controller_server  
/global_costmap/global_costmap  
/local_costmap/local_costmap  
/map_server  
/planner_server  
/robot_state_publisher  
/lifecycle_manager_navigation  
/lifecycle_manager_localization  
/v4l2_camera_node
```

실시간 ETA & 카메라 스트리밍



도착 예정 시간 (ETA)

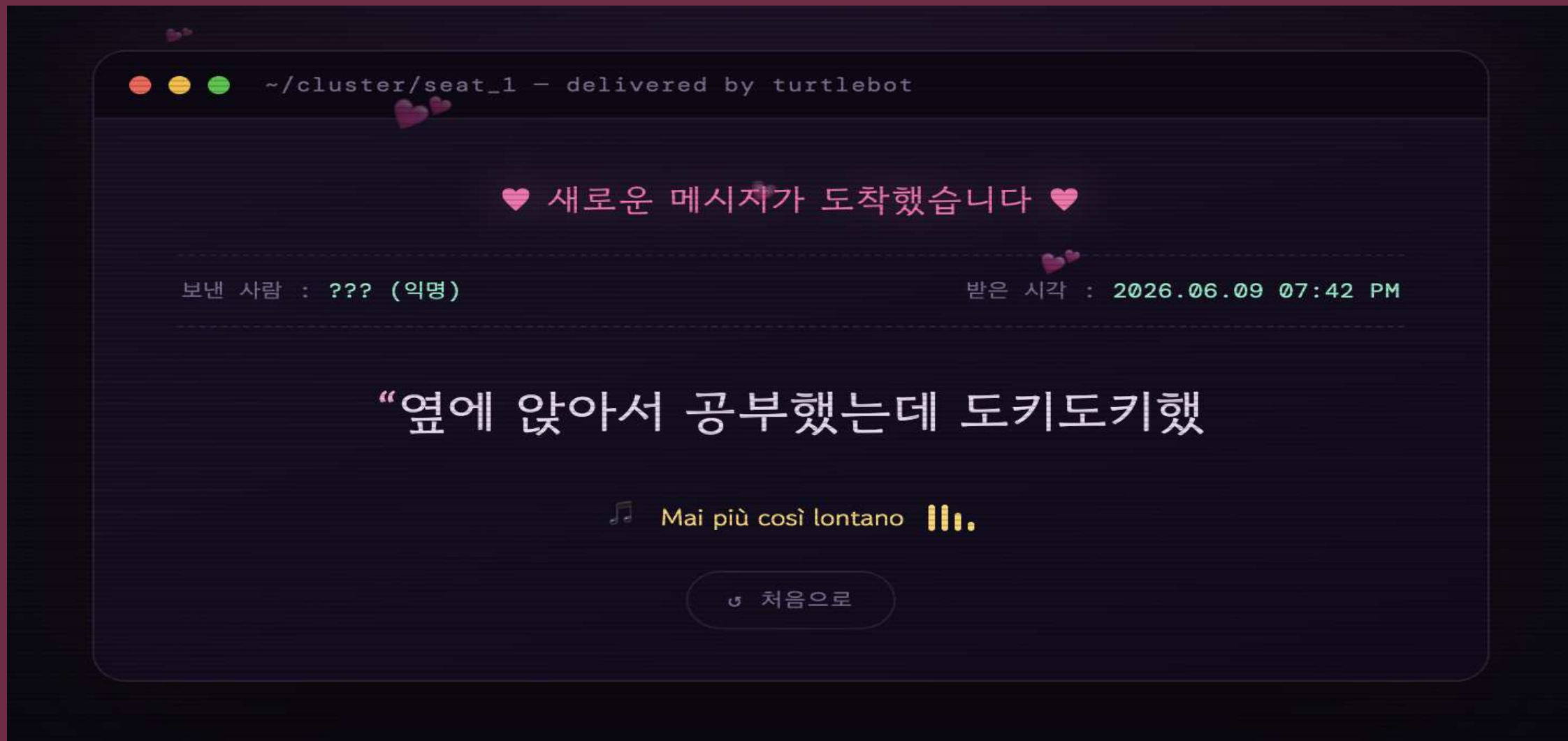
- 로봇으로부터 남은 이동 시간을 받아와 화면에 표시
- 아직 실제 남은 시간이 전달되지 않았을 때는 현재 거리와 로봇 속도를 이용해 예상 시간을 먼저 보여줌
- 실제 남은 시간이 들어오면 바로 반영하여 화면의 카운트다운이 자연스럽게 갱신
- 남은 거리 정보를 이용해 미니맵 위 로봇의 위치도 함께 업데이트



카메라 스트리밍

- 터틀봇 카메라의 이미지 노드에서 발행하는 데이터를 압축 이미지 형식으로 변경하여 사용
- 웹 브릿지가 그 사진을 웹에서 볼 수 있는 형식으로 바꿔서 브라우저에 전달
- 네트워크 부담을 줄이기 위해 1초에 10장으로 속도를 제한함

도착의 순간 — 감성 터미널 UI



실제로 부딪힌 문제들



바닥의 물딩을 넘지 못함

문제 Burger는 지상고가 낮아 문지방·단차를 통과 못함. 라이다는 수평 스캔이라 낮은 단차를 장애물로 인식하지 못해 자꾸 끼임.

해결 물딩 위에 의자를 의도적으로 배치 → 라이다가 의자 다리를 장애물로 감지 → Nav2 costmap이 해당 구역을 회피 경로로 계획해 우회 주행 성공.



맵 스캔에서의 시행착오

문제 NavigateToPose 액션은 경로 계획 완료 후에만 ETA 피드백을 주므로, 목표 전송 직후 1~3초간 브라우저에 공백이 생깁니다.

해결 피드백 첫 수신 전에는 거리 기반 추정값으로 선행 표시.
실제 피드백이 오면 그 값으로 자연스럽게 덮어서 해결.

결과 그리고 한계



검증된 것

- 자율주행으로 지정 좌석까지 충돌 없이 도달
- 실시간 ETA · 미니맵 위치 갱신 정상 동작
- 전 과정 end-to-end 시연 성공



한계 & 발전 방향

- 클러스터 내부가 아닌 외부 공간에서 시연
- 다중 좌석 순차적으로 전달
- 메세지 큐잉
- 외부 디스플레이/스피커 장착

THANK YOU

기술이 사람과 사람 사이에
다리를 놓을 수 있다면.

